



UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO"

DEPARTAMENTO DE CIENCIAS DE LA TECNOLOGÍA E INNOVACIÓN

INGENIERÍA MECATRÓNICA

Caja Fuerte Inteligente con ESP32 y Coprocesamiento con Raspberry Pi 3

INGENIERÍA MECATRÓNICA



Materia: Sistemas Embebidos II (IMT-322)

Estudiante(s): Valeria Alejandra Hoyos Tovar

Docente: Ing. Alan Cornejo

Semestre: 1/2026

Bolivia, 9 de junio de 2026

Índice

1. Descripción Técnica-Conceptual del Proyecto	1
1.1. Contexto y Motivación	1
1.2. Descripción General del Sistema	1
2. Propósito del Proyecto	1
3. Alcance del Proyecto	2
4. Supuestos del Proyecto	3
5. Requerimientos	4
5.1. Requerimientos Funcionales	4
5.2. Requerimientos de Hardware	4
5.3. Requerimientos de Software	5
5.4. Requerimientos de Seguridad	5
5.5. Requerimientos de Documentación	5
6. Componentes de Hardware	5
7. Arquitectura del Sistema	6
7.1. Concepto de Coprocesamiento	6
7.2. Distribución de Tareas	6
8. Protocolo de Comunicación UART	7
8.1. Configuración	7
8.2. Protocolo de Comandos	7
9. Conexiones de Hardware	8
9.1. Teclado Matricial 4×4	8
9.2. Pantalla OLED I2C	8
9.3. Módulo Relé, Buzzer y UART	9
9.4. Circuito del Solenoide	9
10. Firmware del ESP32	9
10.1. Descripción General	9
10.2. Mapa de Pines Utilizados	10
10.3. Lógica de Autenticación	10
10.4. Código – Función de Apertura	10
10.5. Código – Recepción de Comandos UART	11
11. Implementación con ESP-IDF y FreeRTOS	11
11.1. Descripción	11
11.2. Estructura de Módulos	12
11.3. Tareas FreeRTOS	12

12.Servidor Web en Raspberry Pi 3	13
12.1. Descripción General	13
12.2. Configuración de la Raspberry Pi	13
12.3. Funcionalidades de la Interfaz Web	13
12.4. Endpoints del Servidor	14
12.5. Código – Sincronización Bidireccional de Contraseña	14
13.Diseño e Implementación de la PCB	15
13.1. Herramienta de Diseño	15
13.2. Componentes en la PCB	15
13.3. Proceso de Fabricación	15
14.Pruebas Funcionales	16
15.Entregables del Proyecto	17
16.Resultados y Discusión	18
16.1. Resultados Obtenidos	18
16.2. Desafíos Encontrados y Soluciones	19
17.Conclusiones	19

Índice de figuras

Índice de tablas

1.	Lista de componentes del sistema.	6
2.	Distribución de tareas entre ESP32 y Raspberry Pi 3.	7
3.	Configuración del protocolo UART.	7
4.	Protocolo de comandos UART implementado.	8
5.	Conexiones del teclado matricial al ESP32.	8
6.	Conexiones de la pantalla OLED al ESP32.	8
7.	Conexiones del módulo relé, buzzer y UART al ESP32.	9
8.	Resumen de pines GPIO utilizados en el ESP32.	10
9.	Estructura de módulos del código ESP-IDF.	12
10.	Endpoints del servidor web Flask.	14
11.	Conectores incluidos en la PCB.	15
12.	Tabla de pruebas funcionales del sistema.	17
13.	Desafíos técnicos y soluciones implementadas.	19

1. Descripción Técnica-Conceptual del Proyecto

1.1. Contexto y Motivación

En la actualidad, la seguridad física de bienes y documentos importantes representa una necesidad presente tanto en hogares como en pequeñas empresas. Las soluciones comerciales disponibles en el mercado suelen ser costosas y, en su mayoría, carecen de conectividad remota o de capacidades de trazabilidad que permitan conocer quién accedió al sistema y en qué momento. Por otro lado, los sistemas de seguridad de bajo costo basados en microcontroladores simples tampoco ofrecen estas funcionalidades de manera integrada.

El presente proyecto propone el diseño e implementación de una **caja fuerte inteligente basada en coprocesamiento** entre un microcontrolador **ESP32** y una **Raspberry Pi 3**, que integra autenticación por contraseña, interfaz web remota con servidor Flask, comunicación UART bidireccional y registro de eventos en tiempo real. El sistema combina una estructura física metálica con un sistema electrónico de control de acceso avanzado, sin depender de servicios externos en la nube.

1.2. Descripción General del Sistema

El sistema implementa una arquitectura de coprocesamiento donde dos dispositivos embebidos trabajan de forma coordinada y simultánea:

- **ESP32 (unidad de control local):** Gestiona el teclado matricial 4×4, la pantalla OLED I2C, el buzzer activo, el módulo relé y el solenoide de 12V. Es responsable de la lógica de autenticación local y del control mecánico de la cerradura.
- **Raspberry Pi 3 (unidad de control remoto):** Ejecuta un servidor web Flask que permite monitorear el estado del sistema, abrir la caja remotamente con autenticación por contraseña, cambiar credenciales y consultar el historial de eventos. Se comunica con el ESP32 mediante el protocolo UART.

La arquitectura general del sistema puede resumirse como:

[Teclado 4x4] → [ESP32] $UART \rightleftharpoons$ [Raspberry Pi 3] → [Interfaz Web]

2. Propósito del Proyecto

El proyecto se realiza con los siguientes propósitos:

- Demostrar el concepto de **coprocesamiento** mediante la distribución real de tareas entre dos dispositivos embebidos que operan de forma coordinada y simultánea.
- Implementar **comunicación bidireccional** entre un microcontrolador y un computador de placa única mediante el protocolo UART.

- Diseñar y fabricar una **PCB personalizada** utilizando KiCad 9.0 y el método de transferencia térmica con cloruro férrico.
- Proveer un sistema de control de acceso con autenticación por contraseña, bloqueo automático y **sincronización bidireccional de credenciales** entre el control físico y la interfaz web.
- Aplicar conceptos avanzados de sistemas embebidos: manejo de protocolos de comunicación (I2C, UART), programación de servidores web sobre Linux embebido y programación concurrente con FreeRTOS.
- Ofrecer una solución de bajo costo, autónoma y sin dependencia de servicios en la nube, replicable en contextos domésticos o de pequeñas empresas.

3. Alcance del Proyecto

El proyecto incluye:

- Diseño e implementación del firmware completo en ESP32 utilizando Arduino IDE con el SDK de Espressif.
- Implementación adicional del código en **ESP-IDF con FreeRTOS** para demostrar manejo avanzado de tareas concurrentes con drivers nativos.
- Sistema de autenticación por contraseña numérica ingresada mediante teclado matricial 4×4.
- Mecanismo de apertura controlado por solenoide de 12V a través de módulo relé.
- Interfaz local mediante pantalla OLED 0.96" con protocolo I2C.
- Alertas sonoras mediante buzzer activo de 5V para acceso concedido y denegado.
- Servidor web embebido en Raspberry Pi 3 (Flask/Python) que permite:
 - Consultar el estado actual de la caja (abierta, cerrada o bloqueada) en tiempo real.
 - Abrir la caja remotamente con autenticación por contraseña.
 - Revisar el historial de accesos con hora exacta de cada evento.
 - Cambiar la contraseña de acceso verificando primero la contraseña actual.
 - Visualizar el contador de intentos fallidos mediante indicador visual.
- Sistema de bloqueo automático tras 3 intentos fallidos consecutivos con temporizador de desbloqueo de 10 segundos.

- **Sincronización bidireccional de contraseña:** cuando se cambia desde el teclado físico, el ESP32 notifica a la Raspberry Pi; cuando se cambia desde la web, la Raspberry Pi notifica al ESP32.
- PCB personalizada diseñada en KiCad 9.0 y fabricada con método de transferencia térmica.
- Código fuente documentado y publicado en repositorio GitHub.

El presente proyecto no incluye:

- Conexión a internet ni servicios en la nube; el sistema opera exclusivamente en red local.
- Aplicación móvil nativa; el control remoto se realiza mediante navegador web estándar.
- Sistema de alimentación con batería de respaldo ante cortes de energía.
- Notificaciones push, correo electrónico o SMS ante eventos de seguridad.
- Integración con sistemas de alarmá domótica externos.

4. Supuestos del Proyecto

Para el desarrollo del presente proyecto se supone que:

- Se dispone de acceso a un módulo ESP32 WROOM-32 (30 pines) durante todo el período de desarrollo.
- Los componentes electrónicos necesarios pueden adquirirse en el mercado local.
- Existe disponibilidad de una red WiFi local (hotspot o router doméstico) para las pruebas del servidor web.
- El entorno de desarrollo (Arduino IDE con soporte para ESP32 y VS Code con ESP-IDF) puede instalarse y configurarse en la computadora personal disponible.
- Se dispone de herramientas para la fabricación de la PCB: plancha, cloruro férrico, papel fotográfico e impresora láser.
- La evaluación del proyecto considera tanto el correcto funcionamiento del sistema hardware/firmware como la calidad de la documentación técnica entregada.

5. Requerimientos

5.1. Requerimientos Funcionales

- RF-1 El sistema debe autenticar al usuario mediante contraseña numérica de mínimo 4 dígitos ingresada por teclado matricial.
- RF-2 El sistema debe activar el solenoide para abrir la caja únicamente cuando la contraseña sea correcta.
- RF-3 El sistema debe bloquear el acceso durante 10 segundos tras 3 intentos de contraseña incorrectos consecutivos.
- RF-4 El sistema debe mostrar en la pantalla OLED el estado actual en todo momento.
- RF-5 El sistema debe emitir alertas sonoras distintas para acceso concedido y acceso denegado.
- RF-6 El servidor web debe ser accesible desde cualquier dispositivo en la misma red, sin software adicional.
- RF-7 El usuario debe poder cambiar la contraseña desde la web verificando primero la actual.
- RF-8 El usuario debe poder cambiar la contraseña desde el teclado físico presionando la tecla **A**.
- RF-9 Ambas contraseñas (web y teclado) deben sincronizarse automáticamente mediante UART.
- RF-10 El historial de eventos debe mostrar hora exacta y tipo de evento.

5.2. Requerimientos de Hardware

- RH-1 El microcontrolador principal debe ser un ESP32 WROOM-32 con WiFi integrado.
- RH-2 La pantalla debe comunicarse mediante protocolo I2C (dirección 0x3C).
- RH-3 La comunicación entre ESP32 y Raspberry Pi debe realizarse mediante UART (TX2/RX2).
- RH-4 El relé debe operar a 5V con separación galvánica para proteger el ESP32.
- RH-5 El solenoide debe operarse con fuente externa de 12V a través del relé.
- RH-6 Todos los dispositivos deben compartir referencia común de tierra (GND).

5.3. Requerimientos de Software

- RS-1 El firmware del ESP32 debe implementarse con manejo de múltiples periféricos simultáneos.
- RS-2 El código fuente debe estar estructurado en módulos separados por funcionalidad.
- RS-3 El servidor web debe responder en menos de 2 segundos ante cualquier solicitud.
- RS-4 El protocolo UART debe implementar comandos específicos documentados para cada acción.
- RS-5 La interfaz web debe actualizarse en tiempo real mediante polling cada 1 segundo.

5.4. Requerimientos de Seguridad

- RSeg-1 La interfaz web debe requerir autenticación por contraseña para cualquier acción crítica.
- RSeg-2 El sistema debe bloquear el acceso tanto desde el teclado físico como desde la web tras 3 intentos fallidos.
- RSeg-3 El cambio de contraseña debe requerir verificación de la contraseña actual.

5.5. Requerimientos de Documentación

- RD-1 Se debe entregar el esquemático completo de conexiones eléctricas.
- RD-2 El código fuente debe incluir comentarios explicativos en cada función principal.
- RD-3 Se debe entregar una memoria técnica del proyecto con resultados de pruebas funcionales.
- RD-4 El código fuente debe publicarse en un repositorio GitHub con README completo.

6. Componentes de Hardware

En la Tabla 1 se presenta la lista completa de componentes utilizados en el proyecto.

Tabla 1: Lista de componentes del sistema.

Componente	Modelo	Cant.	Función
Microcontrolador	ESP32 WROOM-32 (30 pines)	1	Control local principal
Computador SBC	Raspberry Pi 3 Model B	1	Servidor web / control remoto
Pantalla	OLED 0.96" I2C SSD1306	1	Visualización de estado
Teclado	Matricial 4×4 membrana	1	Entrada de contraseña
Relé	Módulo 2 canales 5V	1	Control del solenoide
Cerradura	Solenoide 12V	1	Mecanismo de apertura
Buzzer	Buzzer activo 5V	1	Alertas sonoras
Fuente	Adaptador DC 12V 1A	1	Alimentación solenoide
Fuente	Cargador USB 5V 2A	1	Alimentación ESP32
PCB	Placa cobre virgen FR4	1	Circuito impreso personalizado
Diodos	1N4007	2	Protección contracorriente

7. Arquitectura del Sistema

7.1. Concepto de Coprocesamiento

El coprocesamiento es la distribución de responsabilidades entre dos o más procesadores que trabajan coordinadamente. En este proyecto, ninguno de los dos dispositivos puede reemplazar al otro de forma eficiente:

- El **ESP32** maneja hardware en tiempo real (teclado, OLED, relé, buzzer) con latencias en el orden de milisegundos.
- La **Raspberry Pi 3** ejecuta un sistema operativo Linux completo que permite correr un servidor web Flask, manejar múltiples conexiones HTTP simultáneas y mantener un historial de eventos.

7.2. Distribución de Tareas

En la Tabla 2 se muestra cómo se distribuyen las responsabilidades entre ambos dispositivos.

Tabla 2: Distribución de tareas entre ESP32 y Raspberry Pi 3.

ESP32 (Control local)	Raspberry Pi 3 (Control remoto)
Lectura del teclado matricial	Ejecución del servidor web Flask
Visualización en pantalla OLED	Autenticación de usuarios web
Control del relé y solenoide	Historial de eventos con hora exacta
Alertas sonoras con buzzer	Cambio de contraseña vía web
Lógica de autenticación local	Sincronización de contraseña
Comunicación UART con RPi	Comunicación UART con ESP32
Bloqueo por intentos fallidos	Visualización de estado en tiempo real

8. Protocolo de Comunicación UART

8.1. Configuración

La comunicación entre el ESP32 y la Raspberry Pi 3 se implementa mediante el protocolo **UART** con la configuración mostrada en la Tabla 3.

Tabla 3: Configuración del protocolo UART.

Parámetro	Valor
Velocidad (Baud Rate)	9600 bps
Bits de datos	8 bits
Bits de parada	1 bit
Paridad	Ninguna
Control de flujo	Ninguno
Pin TX del ESP32	GPIO17 (TX2)
Pin RX del ESP32	GPIO16 (RX2)
Pin TX de la Raspberry Pi	Pin 8 (GPIO14)
Pin RX de la Raspberry Pi	Pin 10 (GPIO15)
Puerto serie Raspberry Pi	/dev/ttyS0

8.2. Protocolo de Comandos

En la Tabla 4 se documentan todos los comandos del protocolo UART implementado.

Tabla 4: Protocolo de comandos UART implementado.

Comando	Dirección	Origen	Descripción
OPEN	RPi $\rightarrow$ ESP32	Interfaz web	Orden de apertura de la caja
PASS:xxxx	RPi $\rightarrow$ ESP32	Interfaz web	Cambio de contraseña desde web
ABIERTO	ESP32 $\rightarrow$ RPi	Control local	Notificación de apertura
CERRADO	ESP32 $\rightarrow$ RPi	Control local	Notificación de cierre
BLOQUEADO	ESP32 $\rightarrow$ RPi	Control local	Sistema bloqueado por intentos
DESBLOQUEADO	ESP32 $\rightarrow$ RPi	Control local	Sistema desbloqueado
PASS_OK	ESP32 $\rightarrow$ RPi	Control local	Contraseña cambiada exitosamente
NEWPASS:xxxx	ESP32 $\rightarrow$ RPi	Teclado físico	Nueva contraseña desde teclado

9. Conexiones de Hardware

9.1. Teclado Matricial 4 $\times$ 4

Tabla 5: Conexiones del teclado matricial al ESP32.

Pin Teclado	GPIO ESP32	Pin Teclado	GPIO ESP32
R1 (Fila 1)	GPIO13	C1 (Col 1)	GPIO26
R2 (Fila 2)	GPIO12	C2 (Col 2)	GPIO25
R3 (Fila 3)	GPIO14	C3 (Col 3)	GPIO33
R4 (Fila 4)	GPIO27	C4 (Col 4)	GPIO32

9.2. Pantalla OLED I2C

Tabla 6: Conexiones de la pantalla OLED al ESP32.

Pin OLED	Pin ESP32
VCC	3.3V
GND	GND
SDA	GPIO21
SCL	GPIO22

9.3. Módulo Relé, Buzzer y UART

Tabla 7: Conexiones del módulo relé, buzzer y UART al ESP32.

Componente / Pin	Pin ESP32
Relé – VCC	VIN (5V)
Relé – GND	GND
Relé – IN1	GPIO18
Buzzer – Positivo (+)	GPIO5
Buzzer – Negativo (–)	GND
UART TX2	GPIO17 → Raspberry Pi Pin 10
UART RX2	GPIO16 → Raspberry Pi Pin 8
GND común	GND → Raspberry Pi Pin 6

9.4. Circuito del Solenoide

El solenoide opera a 12V y se controla a través del módulo relé. Los diodos 1N4007 se colocan en paralelo con el solenoide para protección contra corrientes de contracorriente (flyback):

- Terminal **COM** del relé → Positivo (+) de la fuente 12V
- Terminal **NO** del relé → Cable positivo del solenoide
- Cable negativo del solenoide + GND fuente → GND común del sistema

10. Firmware del ESP32

10.1. Descripción General

El firmware del ESP32 fue desarrollado en **Arduino IDE 2.3.9** utilizando la versión 2.0.17 del framework de Espressif para ESP32. El código implementa la lógica de autenticación local, el control de todos los periféricos y la comunicación UART bidireccional con la Raspberry Pi.

10.2. Mapa de Pines Utilizados

Tabla 8: Resumen de pines GPIO utilizados en el ESP32.

GPIO	Función	Protocolo
GPIO5	Buzzer activo	Digital OUT
GPIO12, 13, 14, 27	Filas del teclado	Digital OUT
GPIO16 (RX2)	UART recepción	UART2
GPIO17 (TX2)	UART transmisión	UART2
GPIO18	Control relé	Digital OUT
GPIO21 (SDA)	Bus I2C maestro	I2C
GPIO22 (SCL)	Bus I2C maestro	I2C
GPIO25, 26, 32, 33	Columnas del teclado	Digital IN

10.3. Lógica de Autenticación

El sistema implementa la siguiente lógica de autenticación local:

1. El usuario ingresa dígitos mediante el teclado. Cada dígito se muestra como “*” en la pantalla OLED.
2. Al presionar #, el sistema compara la entrada con la contraseña almacenada.
3. Si es correcta: activa el relé por 5 segundos, emite tono de éxito y notifica “ABIERTO” a la Raspberry Pi.
4. Si es incorrecta: incrementa el contador, emite tono de error, muestra el número de intento.
5. Al llegar a 3 intentos fallidos: bloquea por 10 segundos, notifica “BLOQUEADO” a la Raspberry Pi.
6. Al presionar *: borra la entrada actual.
7. Al presionar A: entra en modo cambio de contraseña (requiere contraseña actual).

10.4. Código – Función de Apertura

```
1 void abrirCaja() {
2     // Activar solenoide (logica inversa: LOW = activado)
3     digitalWrite(RELAY_PIN, LOW);
4
5     // Emitir tono de acceso concedido (1000 Hz, 500 ms)
6     tone(BUZZER_PIN, 1000, 500);
7
8     // Notificar estado a Raspberry Pi via UART2
9     SerialRasp.println("ABIERTO");
10 }
```

```

11 // Mostrar mensaje en pantalla OLED
12 mostrarMensaje("** ABIERTO **", "Bienvenido!");
13
14 // Mantener solenoide activado por 5 segundos
15 delay(5000);
16
17 // Cerrar la caja y notificar
18 digitalWrite(RELAY_PIN, HIGH);
19 SerialRasp.println("CERRADO");
20 mostrarMensaje("Caja Fuerte", "Ingrese clave:");
21
22 // Reiniciar variables de sesion
23 input = "";
24 intentos = 0;
25 }

```

Listing 1: Función de apertura de la caja fuerte en Arduino IDE.

10.5. Código – Recepción de Comandos UART

```

1 // En el loop principal
2 if (SerialRasp.available()) {
3     String cmd = SerialRasp.readStringUntil('\n');
4     cmd.trim();
5
6     if (cmd == "OPEN") {
7         // Apertura remota desde interfaz web
8         abrirCaja();
9     } else if (cmd.startsWith("PASS:")) {
10        // Cambio de contraseña desde la web
11        password = cmd.substring(5);
12        mostrarMensaje("Clave cambiada!", "Desde web");
13        SerialRasp.println("PASS_OK");
14        delay(2000);
15        mostrarMensaje("Caja Fuerte", "Ingrese clave:");
16    }
17 }

```

Listing 2: Recepción y procesamiento de comandos UART desde la Raspberry Pi.

11. Implementación con ESP-IDF y FreeRTOS

11.1. Descripción

Como implementación avanzada, el proyecto incluye el código equivalente utilizando el **framework nativo de Espressif (ESP-IDF)** con **FreeRTOS**. A diferencia de Arduino IDE, ESP-IDF usa los drivers nativos del hardware y permite organizar el código en tareas concurrentes independientes.

11.2. Estructura de Módulos

Tabla 9: Estructura de módulos del código ESP-IDF.

Archivo	Responsabilidad
main.c	Punto de entrada, inicialización y tarea principal
modules/gpio_control.c	Control del relé y buzzer con drivers GPIO nativos
modules/teclado.c	Lectura del teclado con cola FreeRTOS
modules/oled.c	Comunicación I2C maestro con OLED
modules/uart_comm.c	Comunicación UART2 con Raspberry Pi
modules/caja_fuerte.c	Lógica principal del sistema de seguridad

11.3. Tareas FreeRTOS

```
1 void app_main(void) {
2     ESP_LOGI(TAG, "=== Caja Fuerte Inteligente ===");
3
4     // Inicializar modulos de hardware
5     gpio_control_init();
6     oled_init();
7     uart_comm_init();
8
9     // Cola FreeRTOS para comunicacion entre tareas
10    key_queue = xQueueCreate(10, sizeof(char));
11    teclado_init(key_queue);
12
13    // Crear tareas concurrentes con prioridades distintas
14    // Prioridad 5: teclado (mayor urgencia, tiempo real)
15    // Prioridad 4: uart (comunicacion con RPi)
16    // Prioridad 3: main (logica de negocio)
17    xTaskCreate(teclado_task, "teclado_task", 2048, NULL, 5, NULL);
18    xTaskCreate(uart_task, "uart_task", 2048, NULL, 4, NULL);
19    xTaskCreate(main_task, "main_task", 4096, NULL, 3, NULL);
20
21    ESP_LOGI(TAG, "Sistema listo.");
22 }
```

Listing 3: Creación de tareas concurrentes con FreeRTOS en ESP-IDF.

Las tres tareas operan concurrentemente:

- **teclado_task (prioridad 5):** Lee el teclado matricial cada 20ms y envía teclas detectadas a la cola FreeRTOS.
- **uart_task (prioridad 4):** Monitorea el puerto UART2 y procesa comandos recibidos desde la Raspberry Pi.
- **main_task (prioridad 3):** Procesa las teclas de la cola y ejecuta la lógica de autenticación y control.

12. Servidor Web en Raspberry Pi 3

12.1. Descripción General

El servidor web fue desarrollado en **Python 3** utilizando el framework **Flask 3.1.3**. Se ejecuta en la Raspberry Pi 3 con Raspberry Pi OS Lite y es accesible desde cualquier dispositivo conectado a la misma red mediante el puerto 5000.

12.2. Configuración de la Raspberry Pi

1. **Sistema operativo:** Raspberry Pi OS Lite (64-bit), instalado mediante Raspberry Pi Imager.
2. **Acceso remoto:** SSH habilitado, configurado en modo headless (sin monitor).
3. **Puerto serial:** Habilitado mediante `raspi-config` → `Interface Options` → `Serial Port`.
4. **Entorno virtual Python:** Creado con `python3 -m venv valeria --system-site-packages`.
5. **Dependencias:** Flask 3.1.3 y pyserial instalados en el entorno virtual.

12.3. Funcionalidades de la Interfaz Web

- **Estado en tiempo real:** Muestra el estado actual (ABIERTO, CERRADO, BLOQUEADO) actualizándose cada segundo mediante polling HTTP al endpoint `/estado`.
- **Apertura remota:** Permite abrir la caja ingresando la contraseña correcta. Bloquea la interfaz tras 3 intentos fallidos.
- **Cambio de contraseña:** Requiere ingresar la contraseña actual, la nueva y su confirmación. Sincroniza automáticamente con el ESP32 vía UART.
- **Historial de eventos:** Muestra los últimos 10 eventos con hora exacta.
- **Indicador visual de intentos:** Tres círculos que se llenan de rojo con cada intento fallido.

12.4. Endpoints del Servidor

Tabla 10: Endpoints del servidor web Flask.

Ruta	Método	Descripción
/	GET	Página principal con la interfaz completa
/estado	GET	Retorna estado actual e historial en JSON
/abrir	POST	Verifica contraseña y envía OPEN al ESP32
/cambiar_pass	POST	Verifica contraseña actual y actualiza ambos sistemas

12.5. Código – Sincronización Bidireccional de Contraseña

```
1 def leer_uart():
2     global estado_actual, password_web
3     while True:
4         try:
5             if uart and uart.in_waiting:
6                 linea = uart.readline().decode('utf-8').strip()
7
8                 # Estados del sistema
9                 if linea in ["ABIERTO", "CERRADO",
10                             "BLOQUEADO", "DESBLOQUEADO", "PASS_OK"]:
11                     estado_actual = linea
12                     hora = datetime.now().strftime("%H:%M:%S")
13                     historial.insert(0, {"hora": hora, "evento": linea})
14                     if len(historial) > 10:
15                         historial.pop()
16
17                 # Nueva contraseña desde el teclado físico
18                 elif linea.startswith("NEWPASS:"):
19                     nueva = linea.split(":")[1]
20                     password_web = nueva # Sincronizar en la web
21                     estado_actual = "PASS_OK"
22
23         except Exception:
24             pass
25
26 # Hilo daemon para no bloquear el servidor
27 hilo = threading.Thread(target=leer_uart, daemon=True)
28 hilo.start()
```

Listing 4: Hilo de lectura UART para sincronización de contraseña desde el teclado.

13. Diseño e Implementación de la PCB

13.1. Herramienta de Diseño

La placa de circuito impreso fue diseñada utilizando **KiCad 9.0**. El proceso de diseño incluyó las siguientes etapas:

1. **Diseño del esquemático:** Creación del esquema eléctrico completo con el ESP32 y todos los conectores. Se utilizaron símbolos estándar de la biblioteca de KiCad y el símbolo ESP32-WROOM-32 de la librería RF_Module.
2. **Asignación de huellas:** Asignación de la huella física PinHeader_1xN_P2.54mm_Vertical a cada conector, y headers dobles 2×15 para el ESP32.
3. **Verificación ERC:** Ejecución del verificador de reglas eléctricas, obteniendo **0 errores**.
4. **Generación del PCB:** Transferencia del esquema al editor de PCB mediante “Update PCB from Schematic”.
5. **Ruteo de pistas:** Trazado manual en la capa F.Cu con grosor de 0.8mm.
6. **Plano de tierra:** Creación de zona de relleno GND en la capa B.Cu.
7. **Verificación DRC:** **0 errores** al final del diseño.

13.2. Componentes en la PCB

Tabla 11: Conectores incluidos en la PCB.

Referencia	Tipo de Huella	Conexión
J_OLED	PinHeader 1×4 P2.54mm	Pantalla OLED I2C
J_TECLADO	PinHeader 1×8 P2.54mm	Teclado matricial 4×4
J_RELAY	PinHeader 1×4 P2.54mm	Módulo relé 2 canales
J_RASP	PinHeader 1×4 P2.54mm	Raspberry Pi (UART + GND)
J_BUZZER	PinHeader 1×2 P2.54mm	Buzzer activo
J_POWER	PinHeader 1×2 P2.54mm	Alimentación 5V
U1 (ESP32)	Headers 2×15 P2.54mm	Microcontrolador principal

13.3. Proceso de Fabricación

La PCB fue fabricada mediante el **método de transferencia térmica**, siguiendo los pasos:

1. **Exportación:** El diseño de la capa F.Cu se exportó como PDF en escala 1:1 con opción “Reflejar” activada (espejo horizontal).

2. **Impresión láser:** Se imprimió en papel fotográfico brillante con impresora láser.
3. **Transferencia térmica:** El papel se plancheó sobre la placa de cobre virgen a alta temperatura por 5 minutos.
4. **Remoción del papel:** Inmersión en agua caliente para retirar el papel, dejando el tóner sobre el cobre.
5. **Grabado:** Inmersión en solución de cloruro férrico (FeCl_3) hasta eliminar el cobre expuesto.
6. **Limpieza:** Retiro del tóner con acetona (quitaesmalte de uñas), revelando las pistas de cobre.
7. **Perforación:** Agujeros para los pines de los conectores.

14. Pruebas Funcionales

En la Tabla 12 se documentan las pruebas funcionales realizadas y sus resultados.

Tabla 12: Tabla de pruebas funcionales del sistema.

ID	Prueba	Resultado esperado	Estado
P-01	Contraseña correcta por teclado	Caja abre, OLED muestra ABIERTO	OK
P-02	Contraseña incorrecta	Error en OLED, contador sube	OK
P-03	3 intentos fallidos	Sistema bloqueado 10 segundos	OK
P-04	Tecla * borra entrada	Input limpio, OLED reinicia	OK
P-05	Cambio de clave desde teclado (A)	Nueva clave funciona, RPi sincronizada	OK
P-06	Detección OLED I2C	i2cdetect muestra 0x3C	OK
P-07	Apertura remota desde web	Solenoide activa, estado ABIERTO	OK
P-08	Cambio de clave desde web	ESP32 actualiza clave, PASS_OK	OK
P-09	Estado en tiempo real	Actualiza cada segundo	OK
P-10	Historial de eventos	Eventos con hora exacta	OK
P-11	Bloqueo web tras 3 intentos	Indicador visual lleno, error	OK
P-12	Comunicación UART bidireccional	Comandos y respuestas correctas	OK

15. Entregables del Proyecto

Los entregables del proyecto son:

- Firmware completo del ESP32 (Arduino IDE) con código fuente documentado.
- Implementación adicional en ESP-IDF con FreeRTOS, estructurada en módulos.
- Código del servidor web Flask para Raspberry Pi.
- Esquemático de conexiones eléctricas diseñado en KiCad 9.0.
- Diseño del PCB en KiCad 9.0 con verificación DRC (0 errores).

- Placa PCB fabricada mediante transferencia térmica.
- Prototipo físico funcional del sistema de caja fuerte inteligente.
- Memoria técnica del proyecto (presente documento).
- Tabla de pruebas funcionales con resultados verificados.
- Repositorio GitHub con README completo y diagrama de arquitectura.
- Presentación del proyecto ante el docente.

16. Resultados y Discusión

16.1. Resultados Obtenidos

El sistema de caja fuerte inteligente fue implementado y probado exitosamente, demostrando el correcto funcionamiento de todos los componentes y subsistemas:

- El **coprocesamiento** funcionó correctamente, con el ESP32 manejando hardware en tiempo real y la Raspberry Pi gestionando el servidor web simultáneamente sin interferencias.
- La **comunicación UART bidireccional** se verificó con todos los comandos del protocolo documentado funcionando correctamente.
- La **sincronización de contraseña** funcionó en ambas direcciones: cambios desde el teclado físico se reflejan en la web y viceversa.
- La **interfaz web** es accesible desde cualquier dispositivo con navegador, con actualización de estado cada segundo.
- La **PCB** fue diseñada con 0 errores DRC y fabricada exitosamente.

16.2. Desafíos Encontrados y Soluciones

Tabla 13: Desafíos técnicos y soluciones implementadas.

Desafío	Solución
Conflicto I2C maestro/esclavo en ESP32	Migración a UART para comunicación inter-dispositivo, manteniendo I2C solo para el OLED
Incompatibilidad de librerías Adafruit con ESP32 v3.3.8	Degradación a ESP32 framework v2.0.17 y Adafruit SSD1306 v2.4.6
Mapeo incorrecto del teclado matricial	Ajuste del array <code>keys[ROWS][COLS]</code> mediante prueba empírica con Serial Monitor
Configuración WiFi de Raspberry Pi	Edición manual del archivo <code>network-config</code> en la partición <code>bootfs</code>
Puerto serial incorrecto en Raspberry Pi	Uso de <code>/dev/ttyS0</code> en lugar de <code>/dev/ttyAMA0</code>

17. Conclusiones

El desarrollo del presente proyecto permitió alcanzar los siguientes aprendizajes y conclusiones:

1. El **coprocesamiento** es una estrategia efectiva para sistemas embebidos complejos, distribuyendo responsabilidades entre dispositivos especializados que mejoran el rendimiento y la organización del sistema.
2. La **comunicación UART** resultó ser la opción más adecuada para la comunicación entre el ESP32 y la Raspberry Pi, permitiendo comunicación bidireccional confiable sin los conflictos del bus I2C compartido.
3. El uso de **KiCad 9.0** para el diseño de la PCB permitió obtener un diseño profesional y verificado. El método de fabricación por transferencia térmica es viable para prototipos funcionales de una sola capa.
4. La comparación entre **Arduino IDE y ESP-IDF** demuestra que Arduino IDE es más accesible para desarrollo rápido, mientras que ESP-IDF con FreeRTOS ofrece mayor control del hardware y es el enfoque preferido en sistemas embebidos profesionales.
5. La **sincronización bidireccional de credenciales** es un aspecto crítico en sistemas de seguridad distribuidos, requiriendo un protocolo de comunicación bien definido y documentado para garantizar la consistencia del sistema.